

# Simulated Annealing

Metaheurística basada en el enfriamiento de materiales

Prof. Dr. BARSEKH-ONJI Aboud

Facultad de Ingeniería  
Universidad Anáhuac México

18 de marzo de 2026

# Contenido General

- 1 Motivación y Contexto
- 2 Fundamentos Teóricos
- 3 Pseudocódigo y Algoritmo
- 4 Función de Prueba: Ackley
- 5 Implementación MATLAB
- 6 Análisis de Resultados
- 7 Ventajas, Limitaciones y Variantes
- 8 Conclusiones

# Agenda

- 1 Motivación y Contexto
- 2 Fundamentos Teóricos
- 3 Pseudocódigo y Algoritmo
- 4 Función de Prueba: Ackley
- 5 Implementación MATLAB
- 6 Análisis de Resultados
- 7 Ventajas, Limitaciones y Variantes
- 8 Conclusiones

# ¿Por qué necesitamos metaheurísticas?

## El problema de la optimización compleja

Muchos problemas de ingeniería presentan:

- Espacios de búsqueda **no convexos** con múltiples mínimos locales
- Funciones objetivo **no diferenciables** o de cómputo costoso
- Alta dimensionalidad que hace inviables los métodos exactos

## Limitación de métodos clásicos

El descenso por gradiente y otros métodos locales **quedan atrapados** en mínimos locales. No pueden explorar globalmente el espacio de soluciones.

# Inspiración de la Naturaleza

## Analogía con la metalurgia

El **recocido** (*annealing*) es un proceso térmico donde:

- 1 El material se calienta a alta temperatura
- 2 Los átomos adquieren **alta energía** y se reorganizan libremente
- 3 Se enfría **lentamente y de forma controlada**
- 4 Los átomos llegan a un estado de mínima energía (estructura cristalina óptima)

## Origen del algoritmo

Propuesto por **Kirkpatrick, Gelatt y Vecchi** (1983) en el artículo:  
*"Optimization by Simulated Annealing"*, *Science*, Vol. 220.

Inspirado en la simulación de Monte Carlo de Metrópolis (1953).

# Agenda

- 1 Motivación y Contexto
- 2 Fundamentos Teóricos
- 3 Pseudocódigo y Algoritmo
- 4 Función de Prueba: Ackley
- 5 Implementación MATLAB
- 6 Análisis de Resultados
- 7 Ventajas, Limitaciones y Variantes
- 8 Conclusiones

# El Criterio de Metrópolis

El núcleo matemático del algoritmo se basa en la **distribución de Boltzmann**:

$$P(\Delta E, T) = \exp\left(\frac{-\Delta E}{T}\right) \quad (1)$$

- $\Delta E = f(x_{\text{nuevo}}) - f(x_{\text{actual}})$ : diferencia de costo entre soluciones
- $T$ : temperatura actual del sistema (parámetro de control)
- $P$ : probabilidad de **aceptar una solución peor**

## Propiedad clave

Cuando  $T$  es **alto**,  $P \rightarrow 1$ : se aceptan soluciones malas con frecuencia (exploración).  
Cuando  $T$  es **bajo**,  $P \rightarrow 0$ : solo se aceptan mejoras (explotación).

# Esquema de Enfriamiento

La temperatura disminuye iterativamente mediante un **factor de enfriamiento**  $\alpha$ :

$$T_{k+1} = \alpha \cdot T_k, \quad 0 < \alpha < 1 \quad (2)$$

## Parámetros de control

- $T_0$ : Temperatura inicial (alta)
- $T_{\text{mín}}$ : Temperatura mínima (criterio de paro)
- $\alpha$ : Factor de enfriamiento (0,90–0,99)
- $N_{\text{máx}}$ : Iteraciones máximas

## Impacto de $\alpha$

- $\alpha$  pequeño ( $< 0,9$ ): enfriamiento rápido, riesgo de mínimo local
- $\alpha$  grande ( $> 0,99$ ): enfriamiento lento, mejor exploración pero mayor costo computacional



# Balance entre Exploración y Explotación

## Exploración (*Exploration*):

- Temperaturas altas ( $T$  grande)
- Se aceptan soluciones peores
- El algoritmo “escapa” de mínimos locales
- Mayor diversidad en el espacio de búsqueda

## Explotación (*Exploitation*):

- Temperaturas bajas ( $T$  pequeño)
- Solo se aceptan mejoras
- Convergencia hacia el óptimo
- Refinamiento de la solución actual

## Principio fundamental

El éxito de SA reside en **combinar** ambas fases de forma adaptativa mediante la temperatura. El proceso imita la “paciencia térmica” del recocido metalúrgico.

# Agenda

- 1 Motivación y Contexto
- 2 Fundamentos Teóricos
- 3 Pseudocódigo y Algoritmo
- 4 Función de Prueba: Ackley
- 5 Implementación MATLAB
- 6 Análisis de Resultados
- 7 Ventajas, Limitaciones y Variantes
- 8 Conclusiones

# Pseudocódigo del Simulated Annealing

## Algorithm 1 Simulated Annealing

```
1: Inicializar:  $x_0$  (solución inicial),  $T_0$ ,  $T_{\min}$ ,  $\alpha$ ,  $N_{\max}$ 
2:  $x_{\text{mejor}} \leftarrow x_0$ ;  $f_{\text{mejor}} \leftarrow f(x_0)$ 
3:  $T \leftarrow T_0$ ;  $k \leftarrow 0$ 
4: while  $T > T_{\min}$  and  $k < N_{\max}$  do
5:   Generar vecino:  $x_{\text{nuevo}} \leftarrow x_k + \mathcal{N}(0, 1)$ 
6:   Calcular  $\Delta E = f(x_{\text{nuevo}}) - f(x_k)$ 
7:   if  $\Delta E < 0$  then
8:      $x_{k+1} \leftarrow x_{\text{nuevo}}$  ▷ Aceptar mejora
9:     if  $f(x_{\text{nuevo}}) < f_{\text{mejor}}$  then
10:       $x_{\text{mejor}} \leftarrow x_{\text{nuevo}}$ 
11:     end if
12:   else
13:     if  $\text{rand}(0, 1) < \exp(-\Delta E/T)$  then
14:       $x_{k+1} \leftarrow x_{\text{nuevo}}$  ▷ Aceptar con probabilidad
15:     end if
16:   end if
17:    $T \leftarrow \alpha \cdot T$ ;  $k \leftarrow k + 1$ 
18: end while
19: return  $x_{\text{mejor}}$ 
```

# Agenda

- 1 Motivación y Contexto
- 2 Fundamentos Teóricos
- 3 Pseudocódigo y Algoritmo
- 4 Función de Prueba: Ackley**
- 5 Implementación MATLAB
- 6 Análisis de Resultados
- 7 Ventajas, Limitaciones y Variantes
- 8 Conclusiones

# La Función de Ackley

Una de las funciones de referencia más utilizadas en optimización:

$$f(x, y) = -20 \exp\left(-0,2\sqrt{0,5(x^2 + y^2)}\right) - \exp(0,5(\cos 2\pi x + \cos 2\pi y)) + 20 + e \quad (3)$$

## Características

- **Mínimo global:**  $f(0, 0) = 0$
- **Dominio típico:**  $[-5, 5]^2$
- **Superficie:** irregular, con múltiples mínimos locales
- Diseñada para **engañar** a algoritmos que solo explotan localmente

## Desafío para optimizadores

La función tiene una región central casi plana rodeada de “muros” cosénicos. Un optimizador local queda atrapado fuera del óptimo global.

# Implementación MATLAB — Definición de la Función

## Función anónima en MATLAB

Definición compacta usando  $@(x)$  para pasar un vector  $[x_1, x_2]$ :

## Parámetros matemáticos

- $a = 20, b = 0,2, c = 2\pi$
- Estándar de la literatura

```
1 % Definicion de la funcion de Ackley como funcion anonima
2 ackley = @(x) -20 * exp(-0.2 * sqrt(0.5 * (x(1)^2 + x(2)^2))) ...
3             - exp(0.5 * (cos(2 * pi * x(1)) + cos(2 * pi * x(2)))) ...
4             + 20 + exp(1);
5
6 % Generacion de la malla para visualizacion
7 [x_grid, y_grid] = meshgrid(-5:0.1:5, -5:0.1:5);
8 z_grid = arrayfun(@(x, y) ackley([x, y]), x_grid, y_grid);
```

# Agenda

- 1 Motivación y Contexto
- 2 Fundamentos Teóricos
- 3 Pseudocódigo y Algoritmo
- 4 Función de Prueba: Ackley
- 5 Implementación MATLAB**
- 6 Análisis de Resultados
- 7 Ventajas, Limitaciones y Variantes
- 8 Conclusiones

# Parámetros e Inicialización

```
1 % ---- Parametros del algoritmo ----
2 T      = 1000;      % Temperatura inicial (alta -> maxima exploracion)
3 T_min  = 1;         % Temperatura minima (criterio de paro)
4 alpha  = 0.95;      % Factor de enfriamiento geometrico
5 max_iter = 700;     % Numero maximo de iteraciones
6
7 % ---- Solucion inicial ----
8 x = [0.1, 0.1];     % Punto de partida (cercano al origen)
9 best_sol = x;
10 best_cost = ackley(x); % Evaluar costo inicial
11
12 % ---- Vectores de rastreo ----
13 costs = zeros(1, max_iter);
14 trace_x = x(1); trace_y = x(2); trace_z = best_cost;
```



# Parámetros e Inicialización

Nota sobre  $\alpha = 0,95$

Con  $T_0 = 1000$ ,  $T_{\text{mín}} = 1$  y  $\alpha = 0,95$ : el número de iteraciones para enfriar es  $\lceil \log(T_{\text{mín}}/T_0)/\log(\alpha) \rceil \approx 135$  pasos de temperatura.

# Lazo Principal del SA

```
1 for iter = 1:max_iter
2     % 1. Generar solucion vecina (perturbacion gaussiana)
3     new_x      = x + randn(1, 2);
4     new_cost   = ackley(new_x);
5
6     % 2. Criterio de aceptacion
7     if new_cost < best_cost           % Mejora -> aceptar siempre
8         best_sol  = new_x;
9         best_cost = new_cost;
10        x = new_x;
11    else
12        delta_cost = new_cost - ackley(x);
13        if rand < exp(-delta_cost / T) % Criterio de Metropolis
14            x = new_x;
15        end
16    end
```

# Lazo Principal del SA

```
1 % 3. Registrar trayectoria y actualizar temperatura
2 costs(iter) = best_cost;
3 trace_x(end+1) = x(1); trace_y(end+1) = x(2);
4 trace_z(end+1) = ackley(x);
5 T = T * alpha; % Enfriamiento geometrico
6 if T < T_min, break; end % Criterio de paro termico
7 end
```

# Agenda

- 1 Motivación y Contexto
- 2 Fundamentos Teóricos
- 3 Pseudocódigo y Algoritmo
- 4 Función de Prueba: Ackley
- 5 Implementación MATLAB
- 6 Análisis de Resultados**
- 7 Ventajas, Limitaciones y Variantes
- 8 Conclusiones

# Visualización del Proceso de Búsqueda

## Gráfica 3D: Superficie + Trayectoria

- La superficie `surf()` muestra la función de Ackley
- Puntos rojos (`scatter3`) muestran la **trayectoria explorada**
- Se observa cómo al inicio el algoritmo salta ampliamente (alta  $T$ )
- Después converge hacia el mínimo

## Gráfica 2D: Convergencia del costo

- Eje x: número de iteración
- Eje y:  $f_{\text{mejor}}$  en cada iteración
- Curva descendente confirma convergencia
- La curva no es monótonamente decreciente (SA acepta soluciones peores temporalmente)

## Salida esperada en consola

```
Mejor solución encontrada: (0.0023, -0.0041)  
Costo de la mejor solución: 0.0123
```

# Resumen de Parámetros y su Efecto

Cuadro 1: Parámetros clave del Simulated Annealing

| Parámetro              | Símbolo    | Valor en ejemplo | Efecto                                          |
|------------------------|------------|------------------|-------------------------------------------------|
| Temperatura inicial    | $T_0$      | 1000             | Mayor $\rightarrow$ más exploración inicial     |
| Temperatura mínima     | $T_{\min}$ | 1                | Criterio de paro del enfriamiento               |
| Factor de enfriamiento | $\alpha$   | 0.95             | Más cerca de 1 $\rightarrow$ enfriamiento lento |
| Iteraciones máx.       | $N_{\max}$ | 700              | Límite de evaluaciones de la función            |
| Solución inicial       | $x_0$      | (0.1, 0.1)       | Influye en calidad final                        |

# Agenda

- 1 Motivación y Contexto
- 2 Fundamentos Teóricos
- 3 Pseudocódigo y Algoritmo
- 4 Función de Prueba: Ackley
- 5 Implementación MATLAB
- 6 Análisis de Resultados
- 7 Ventajas, Limitaciones y Variantes**
- 8 Conclusiones

# Ventajas y Limitaciones del SA

## Ventajas:

- **Generalidad:** aplica a cualquier problema con función de costo
- **Garantía asintótica:** con enfriamiento suficientemente lento, converge al óptimo global
- **Simplicidad de implementación:** pocos parámetros y código compacto
- **Robusto** ante ruido y discontinuidades

## Limitaciones:

- **Sensibilidad al schedule:** una mala elección de  $\alpha$  o  $T_0$  puede fallar
- **Convergencia lenta** para espacios de alta dimensión
- **No paralelizable** fácilmente (cadena de Markov secuencial)
- No garantiza el óptimo en tiempo finito

## Recomendación práctica

Para problemas de alta dimensión o multiobjetivo, considerar extensiones como **Parallel SA**, **MOSA** (Multi-Objective SA) o su combinación con metaheurísticas poblacionales.



# Variantes Avanzadas del SA

Cuadro 2: Comparación de variantes del Simulated Annealing

| Variante      | Característica principal    | Aplicación                 |
|---------------|-----------------------------|----------------------------|
| SA clásico    | Enfriamiento geométrico     | Optimización mono-objetivo |
| Fast SA (FSA) | Enfriamiento de Cauchy      | Convergencia más rápida    |
| Adaptive SA   | $T$ se ajusta dinámicamente | Problemas dinámicos        |
| MOSA          | Frente de Pareto            | Optimización multiobjetivo |
| Parallel SA   | Múltiples cadenas           | Cómputo paralelo / GPU     |
| SA + Híbrido  | SA + búsqueda local         | Optimización combinatoria  |

# Agenda

- 1 Motivación y Contexto
- 2 Fundamentos Teóricos
- 3 Pseudocódigo y Algoritmo
- 4 Función de Prueba: Ackley
- 5 Implementación MATLAB
- 6 Análisis de Resultados
- 7 Ventajas, Limitaciones y Variantes
- 8 Conclusiones

# Conclusiones

## Puntos clave

- 1 El **Simulated Annealing** es una metaheurística inspirada en el proceso físico de recocido, capaz de escapar de mínimos locales mediante el criterio de **Metrópolis**.
- 2 La temperatura controla el **balance exploración-explotación**: alta  $T$  favorece diversidad; baja  $T$  favorece convergencia.
- 3 Los parámetros ( $T_0$ ,  $\alpha$ ,  $T_{\text{mín}}$ ) deben elegirse con cuidado; su impacto es **determinante** para la calidad de la solución.
- 4 Aplicado a la **función de Ackley**, el SA demuestra su capacidad de encontrar el mínimo global en un paisaje altamente multimodal.

# Referencias

## Referencia fundamental

Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, 220(4598), 671–680.

<https://doi.org/10.1126/science.220.4598.671>